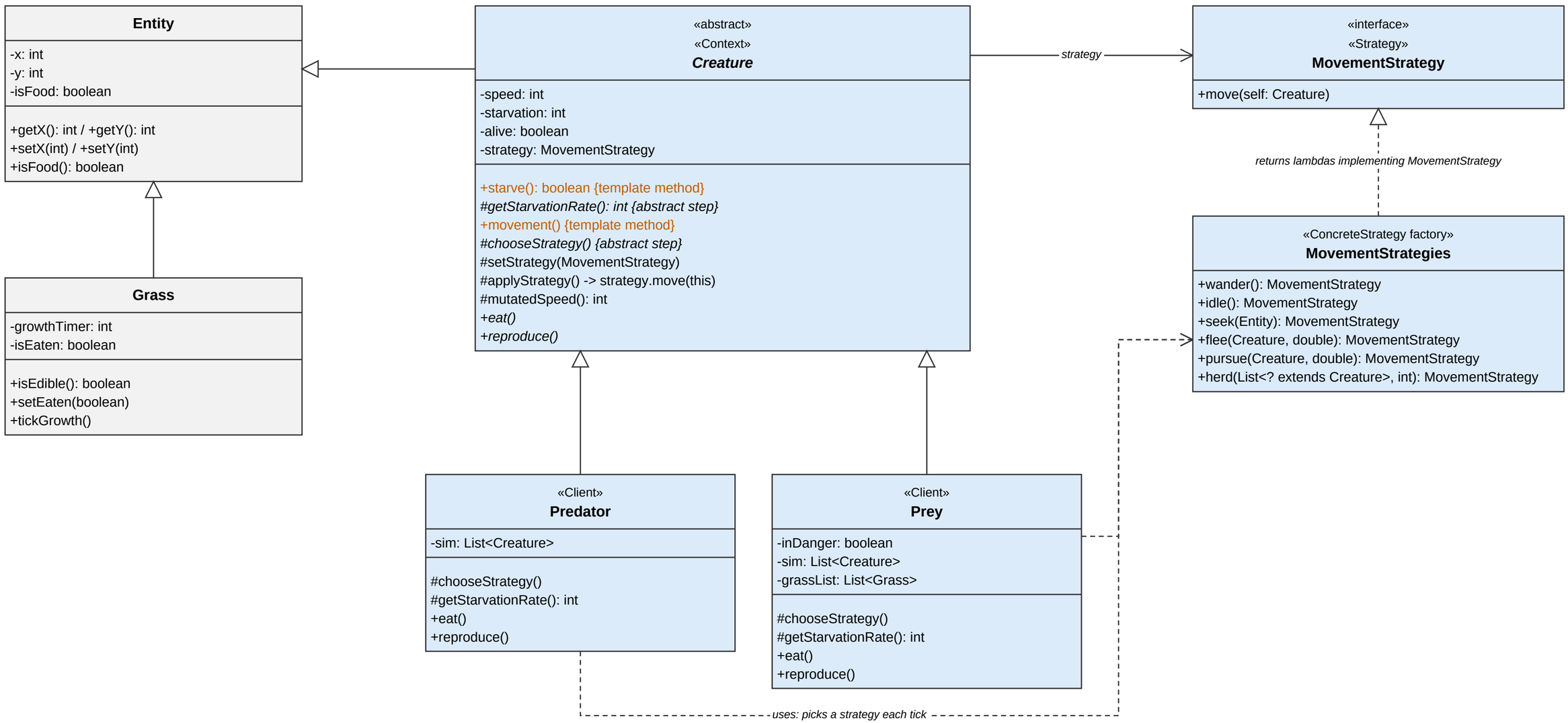
Predator/Prey Simulation - Design Pattern Diagram: Strategy (with Template Method)

Strategy pattern participants

Supporting classes

Orange text = Template Method steps



Strategy - roles

Strategy: MovementStrategy, a functional interface with one method, move(Creature).

Concrete strategies: lambdas returned by MovementStrategies (wander, idle, seek, flee, pursue, herd). Each captures its own target, so no class per behaviour is needed.

Context: Creature holds the current strategy; applyStrategy() delegates to it.

Clients: Predator and Prey choose a strategy at runtime in chooseStrategy().

```
setStrategy(MovementStrategies.flee(threat, FLEE_SPEED_MULTIPLIER));
applyStrategy();
```

Template Method (secondary pattern)

Creature.starve() fixes the algorithm (add the rate plus a metabolic adjustment, then test the threshold) and defers getStarvationRate() to subclasses: Predator returns 3; Prey returns 1, doubled while fleeing.

Creature.movement() fixes the order: chooseStrategy() (abstract step), then applyStrategy().

Game loop (Simulation uses Creature)

Phase 1: snapshot.parallelStream().forEach(Creature::chooseStrategy). Decisions only read shared state and write the creature's own fields.

Phase 2 (sequential): applyStrategy() and eat(), so all world changes stay on one thread.